

Design and Implementation of a Parameterized Barrel Shifter Using Verilog HDL

Verilog Project Report

Sreeja Reddy Nandhikonda
23B81A04B4
ECE – B
BTech 3rd Year 2nd Semester

1.INTRODUCTION

A barrel shifter is an important combinational circuit in digital systems that enables fast shifting and rotation of binary data by a specified number of bit positions within a single clock cycle. It is widely used in processors, arithmetic logic units (ALUs), and signal processing applications where efficient bit manipulation is required. Unlike conventional shift registers that operate sequentially, a barrel shifter provides immediate results, improving overall system performance.

In this project, a parameterized multi-mode barrel shifter is designed using Verilog HDL with an 8-bit implementation. The design supports multiple operations, including logical left shift, logical right shift, arithmetic right shift, and circular rotation in both directions. The use of signed input ensures correct arithmetic shifting with proper sign extension.

The design combines combinational logic for shift operations with a sequential output register controlled by a clock and reset, ensuring reliable synchronous operation. A testbench is developed to verify functionality across different modes and input conditions. Synthesis is performed using timing constraints, and the results indicate that the design meets timing requirements with positive slack.

Power analysis shows that the majority of power consumption is due to switching activity, while leakage power remains minimal. The synthesized schematics further illustrate the hardware implementation of the design using basic digital components.

2. OBJECTIVE

- To design and implement a parameterized multi-mode barrel shifter using Verilog HDL.
- To support multiple operations including Logical left shift, Logical right shift, Arithmetic right shift and Bidirectional circular rotation
- To develop a scalable architecture using parameterization, enabling flexibility in data width (8-bit in this implementation).
- To ensure synchronous operation by incorporating clock-driven output and reset functionality for reliable performance.
- To verify the design using a comprehensive testbench with various input combinations, shift values, and operation modes.
- To apply timing constraints and evaluate the design performance under realistic conditions.
- To analyse the design in terms of timing, power consumption, and hardware implementation after synthesis.
- To understand the complete RTL-to-synthesis design flow for a digital system.

3. TOOLS REQUIRED:

Cadence EDA tools, NC launch Simulator, Genus gui

4. IMPLEMENTATION WORKING

The barrel shifter is implemented using Verilog HDL as a parameterized module, where the data width N can be modified. In this design, an 8-bit configuration is used for simulation and synthesis. A barrel shifter is a combinational circuit that can shift or rotate data by multiple bit positions in a single operation, unlike shift registers which require multiple clock cycles.

Input Signals and Control

data_in - Input data to be shifted
shift - Specifies number of bit positions
mode - Selects the type of operation
rot_dir - Controls rotation direction
clk, rst - Used for synchronous operation

The mode signal enables multi-functionality within a single hardware block, reducing hardware complexity.

Modes of Operation

- mode = 2'b00 - **Logical Left Shift**: Bits are shifted left and zeros are inserted at LSB
- mode = 2'b01 - **Logical Right Shift**: Bits are shifted right and zeros are inserted at MSB
- mode = 2'b10 - **Arithmetic Right Shift**: Preserves the sign bit (MSB) for signed numbers, Implemented using >>> operator
- mode = 2'b11 - **Circular Rotation**: Bits shifted out from one end are reintroduced at the other end. rot_dir = 0 → Left rotation, rot_dir = 1 → Right rotation

Working Principle

- A combinational always @(*) block is used to perform all shift operations.
- A temporary register temp stores intermediate results.
- Rotation is implemented using bitwise shift and OR operations, which mimic circular data movement.
- A special condition is included for shift = 0 to directly pass the input.

Sequential Output Stage

- The output is stored using a clocked register, making the design synchronous.
- On every positive clock edge:
 - If rst is high - output is reset to zero
 - Else - computed value is assigned to output
- This ensures stable and predictable output behavior.

Testbench Verification: A testbench is designed to verify functionality by: Generating a clock signal, applying different input combinations and testing all operation modes

- The \$monitor statement is used to observe real-time simulation results.

Timing Constraints

- Timing constraints are defined to model real hardware conditions: Clock period specified, Input/output delays assigned and Clock uncertainty included
- Reset path is declared as a false path to exclude it from timing analysis.

Synthesis and Hardware Realization

- After synthesis, the design is implemented using Multiplexers, Logic gates, Flip-flops
- Timing analysis shows that the design meets constraints with **positive slack**.
- Power analysis indicates that **switching activity dominates power consumption**, while leakage power is minimal.

CODE:

DESIGN File: barrelshifter.v

```
`timescale 1ns/1ps
module barrel_shifter #(
parameter N = 8)(
input clk,
input rst,
input signed [N-1:0] data_in,
input [$clog2(N)-1:0] shift,
input [1:0] mode,
input rot_dir,
output reg [N-1:0] data_out);
reg [N-1:0] temp;
always @(*) begin
case (mode)
2'b00: temp = data_in << shift;
2'b01: temp = data_in >> shift;
2'b10: temp = data_in >>> shift;
2'b11: begin
if (shift == 0) begin
temp = data_in;
end
else if (rot_dir == 0) begin
temp = (data_in << shift) | (data_in >> (N - shift));
end
else begin
temp = (data_in >> shift) | (data_in << (N - shift));
end
end
end
```

```

    default: temp = data_in;
endcase
end
always @(posedge clk or posedge rst) begin
    if (rst)
        data_out <= 0;
    else
        data_out <= temp;
    end
endmodule

```

TESTBENCH: tb_barrelshifter.v

```

`timescale 1ns/1ps
module tb_barrel_shifter;
    parameter N = 8;
    reg clk;
    reg rst;
    reg [N-1:0] data_in;
    reg [$clog2(N)-1:0] shift;
    reg [1:0] mode;
    reg rot_dir;
    wire [N-1:0] data_out;
    barrel_shifter #(N) uut(.clk(clk), .rst(rst), .data_in(data_in), .shift(shift), .mode(mode),
    .rot_dir(rot_dir), .data_out(data_out));
    always #5 clk = ~clk;
    initial begin
        $display("Time\tMode Rot Shift Input Output");
        clk = 0; rst = 1; #10 rst = 0;
        data_in = 8'b10110011; shift = 2;
        mode = 2'b00; #10;
        data_in = 8'b10110011; shift = 3;
        mode = 2'b01; #10;
        data_in = 8'b10110011; shift = 2;
        mode = 2'b10; #10;
        data_in = 8'b10110011; shift = 1;
        mode = 2'b11; rot_dir = 0; #10;
        data_in = 8'b10110011; shift = 2;
        mode = 2'b11; rot_dir = 1; #10;
        data_in = 8'b11110000; shift = 2;
        mode = 2'b01; #10;
        data_in = 8'b00001111; shift = 2;
        mode = 2'b01; #10;
        data_in = 8'b10000001; shift = 3;
    end
endmodule

```

```

mode = 2'b11; rot_dir = 1; #10;
$finish; end
endmodule

```

SDC File: barrelshiftersdc.sdc

```

create_clock -name clk -period 2 -waveform {0 1} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty -rise 0.01 [get_clocks "clk"]
set_clock_uncertainty -fall 0.01 [get_clocks "clk"]
set_input_delay -max 1.0 -clock [get_clocks "clk"] [get_ports "data_in[*]"]
set_input_delay -max 1.0 -clock [get_clocks "clk"] [get_ports "shift[*]"]
set_input_delay -max 1.0 -clock [get_clocks "clk"] [get_ports "mode[*]"]
set_input_delay -max 1.0 -clock [get_clocks "clk"] [get_ports "rot_dir"]
set_output_delay -max 1.0 -clock [get_clocks "clk"] [get_ports "data_out[*]"]
set_false_path -from [get_ports "rst"]

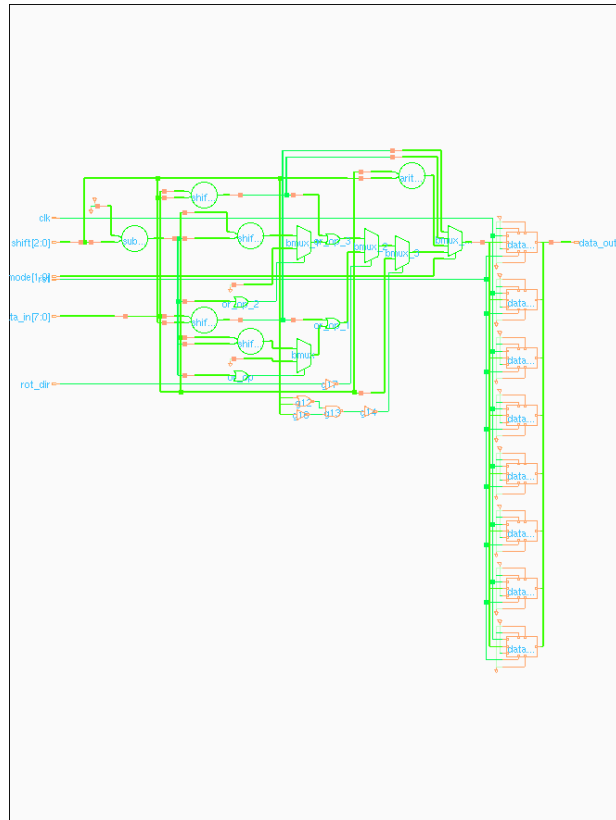
```

RESULTS:

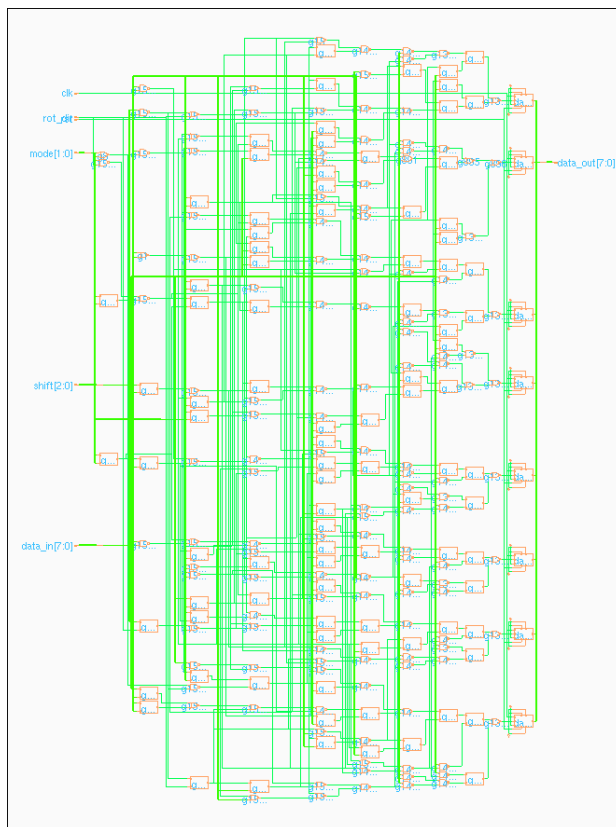
a) Waveform (nclaunch -new)



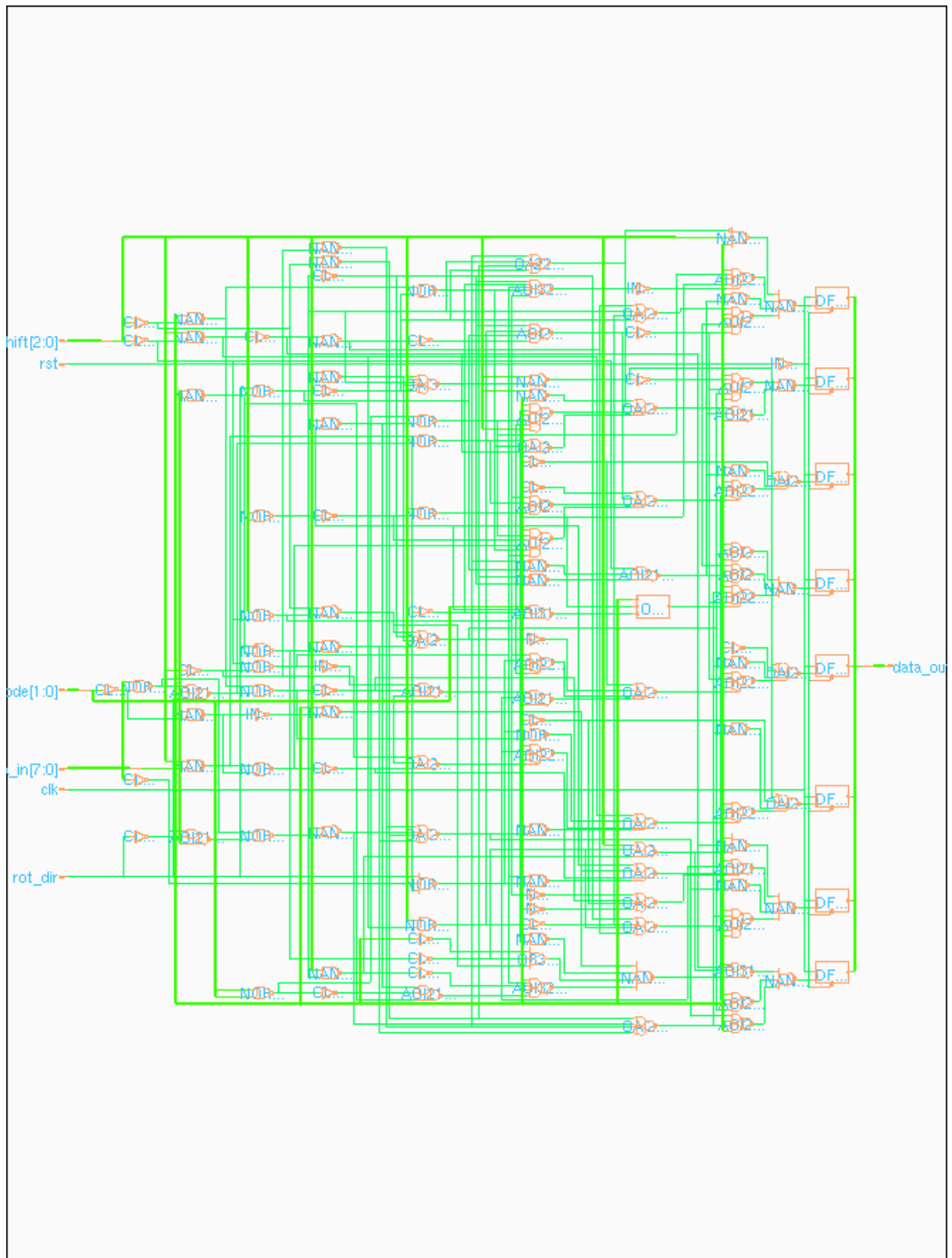
- Initially, `rst = 1`, so `data_out = 0`, shows reset functionality. After reset, output updates only at clock edges, showing synchronous behavior.
- For **mode = 00** with `shift = 2`, input CC produces output **16** - logical left shift.
- For **mode = 01** with `shift = 3`, output becomes **EC** - logical right shift.
- For `mode = 10`, arithmetic right shift preserves the MSB, confirming correct signed operation.
- For `mode = 11`: `rot_dir = 0` - output **67** (left rotation), `rot_dir = 1` - output **3C** (right rotation)
- Changes in input values (e.g., 03, 30) produce correct outputs for corresponding shift values. The signal `temp` updates immediately, while `data_out` updates at clock edges, confirming proper combinational and sequential operation.



(a) RTL Schematic of Barrel Shifter



(b) Optimized Schematic of Barrel Shifter



(c) Gate Level Netlist of Barrel Shifter

Netlist Analysis

- The synthesized netlist represents the gate-level description of the barrel shifter after RTL synthesis.
- It consists of interconnected standard cells such as: Logic gates (AND, OR, INV), Complex gates (AOI, OAI), Flip-flops (DFFs) for output storage
- The shifting and rotation operations are implemented using a network of multiplexer-based logic, which is decomposed into basic gates in the netlist.
- The presence of multiple combinational paths reflects parallel shifting capability, which is a key feature of barrel shifters.
- The control signals (mode, shift, rot_dir) are mapped to selection logic, which determines the data path through the netlist.
- The output (data_out) is driven by flip-flops, confirming that the design is synthesized as a synchronous circuit and Optimization during synthesis results in the use of efficient standard cells, reducing area and improving timing performance.

File: /home/vlsi/siri/synthesis/barrelshifter_power.repo

Page 1 of 1

Instance: /barrel_shifter

Power Unit: W

PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
register	8.03616e-07	9.43146e-05	0.000000e+00	9.51182e-05	51.56%
latch	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
logic	2.14248e-06	4.86820e-05	3.32073e-05	8.40318e-05	45.55%
bbox	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
clock	0.000000e+00	0.000000e+00	5.34600e-06	5.34600e-06	2.90%
pad	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
pm	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
Subtotal	2.94609e-06	1.42997e-04	3.85533e-05	1.84496e-04	100.01%
Percentage	1.60%	77.51%	20.90%	100.00%	100.00%

Power Report

File: /home/vlsi/siri/synthesis/barrelshifter_area.repo

Page 1 of 1

```
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 01 2026 01:30:28 pm
Module:            barrel_shifter
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====
```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
barrel_shifter		142	759.171	0.000	759.171	<none> (D)
(D) = wireload is default in technology library						

Area Report

```

=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:     Apr 01 2026 01:30:18 pm
Module:          barrel_shifter
Operating conditions: slow (balanced_tree)
Wireload mode:   enclosed
Area mode:       timing library
=====

```

Path 1: MET (3 ps) Setup Check with Pin data_out_reg[4]/CK->D

```

Group: clk
Startpoint: (R) mode[0]
Clock: (R) clk
Endpoint: (F) data_out_reg[4]/D
Clock: (R) clk

          Capture      Launch
Clock Edge:+      2000      0
Drv Adjust:+      0        0
Src Latency:+      0        0
Net Latency:+      0 (I)    0 (I)
Arrival:=      2000      0

Setup:-      59
Required Time:= 1941
Launch Clock:- 0
Input Delay:- 1000
Data Path:-    938
Slack:=       3

```

```

Exceptions/Constraints:
input_delay      1000      barrelshiftersdc.sdc_line_11_8_1

```

```

#-----
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival
# Instance
# Location
#-----
#
mode[0]          -      -   R    (arrival)  5 21.9   0   0
1000  (-,-)
g3717/Y          -      A->Y F    CLKINVX3  2  7.1   22  21
1021  (-,-)
g3704_1617/Y     -      B->Y R    NOR2X2   2  8.4   94  78
1099  (-,-)
g3670_2883/Y     -      A->Y F    NOR2X2   6 12.3   69  72
1171  (-,-)
g3665/Y          -      A->Y R    CLKINVX1  5 14.9  132 118
1290  (-,-)
g3640_5477/Y     -      A1->Y F    OAI22X2  7 14.8  158 144
1434  (-,-)
g3633/Y          -      A->Y R    INVXL    1  1.7   67  72
1505  (-,-)
g3599_6131/Y     -      A1->Y F    OAI211XL 1  1.7  136 116
1622  (-,-)
g3597_8428/Y     -      C0->Y R    AOI221XL 1  1.7  179 161
1783  (-,-)
g3592_2398/Y     -      C0->Y F    OAI211XL 1  2.3  164 155
1938  (-,-)
data_out_reg[4]/D <<< -      F    DFFRHQX1  1  -    -   0
1938  (-,-)
#

```

Timing Report

Area Report: The synthesized barrel shifter occupies a total area of 759.17 units with a cell count of 142, while the net area is reported as 0 due to the use of a default wire load model. The area is primarily contributed by the combinational shifting logic and output registers, and the behavioural implementation may not be fully area optimized.

Power Report: The total power consumption of the design is 1.84496×10^{-4} W (≈ 184 μ W), with internal power contributing 77.51%, switching power 20.90%, and leakage power only 1.60%. The registers consume 51.56% of the total power, while the combinational logic contributes 45.55%, indicating that the design is dominated by dynamic power.

Timing Report: The design meets timing with a positive slack of 3 ps, where the required arrival time is 1941 ps and the actual data arrival time is 1938 ps. The critical path exists between flip-flops and passes through the combinational shifting logic, resulting in a very tight timing margin.

CONCLUSION

The project successfully implements a parameterized multi-mode barrel shifter using Verilog HDL, capable of performing logical shifts, arithmetic shifts, and bidirectional rotations efficiently within a single clock cycle. The design demonstrates flexibility through parameterization and correct handling of signed data for arithmetic operations.

Functional verification using the testbench confirms that all modes operate correctly under different input conditions. The synthesized design further validates the implementation, with timing analysis showing positive slack and no violations, and power analysis indicating efficient consumption with minimal leakage.

The generated schematics and netlist confirm accurate hardware realization using standard cells, multiplexers, and flip-flops, reflecting a proper synchronous design. Overall, the project provides a complete understanding of the digital design flow from RTL modelling to synthesis, making it suitable for VLSI applications.